



ОНЛАЙН-ОБРАЗОВАНИЕ

Меня хорошо слышно && видно?



Напишите в чат, если есть проблемы!

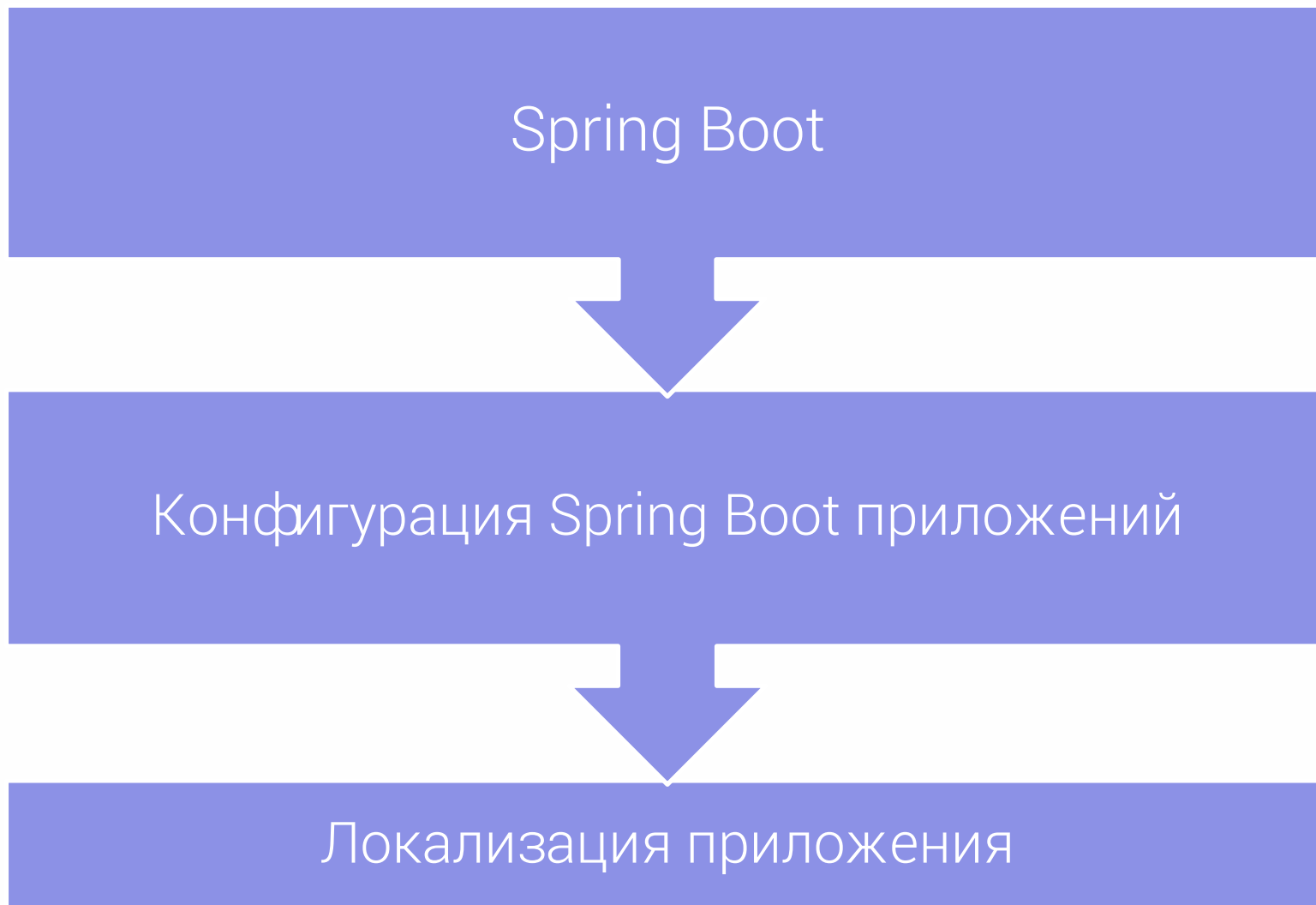
Ставьте ☐ + если все хорошо

Ставьте ☐ - если есть проблемы



Чёрная магия Spring Boot





- ❖ Как пойдет занятие зависит от того, как будут задаваться вопросы
- ❖ Поэтому желательно вопросы задавать конце блока. Для этого у нас есть специальные слайды
- ❖ Но! Если что-то мешает дальнейшему пониманию темы лучше сразу задать вопрос в чат
- ❖ Оффтоп и выходящие за рамки занятия вопросы лучше задать в группе телеграмма . Или в чат зума. Я их сохраню и отвечу после занятия
- ❖ В конце с вас отзыв)



01

Spring Boot

- Разработчиков мало
- Пользователей 1.5 человека (преуменьшение)
- Решения разрабатываются долго и часто “с нуля”
- А требования к ним прорабатываются полностью до начала разработки
- Конкуренция не высокая
- Часто один сервер приложений хостил несколько больших приложений

- Новые фичи и продукты надо выпускать как можно быстрее, конкуренция высокая
- Производство должно быть максимально дешево
- Требования к продукту постоянно меняются, буквально на лету
- Куча разных библиотек и фреймворков
- Каждый как-то конфигурируется и может быть не совместим с другим
- Микросервисы, масштабирование и мониторинг
- Пользователей тьма

Spring Boot – специальный фреймворк для быстрого создания production-grade приложений

- Собственно, Spring Boot – максимально упрощает конфигурирование.



- Stand-alone приложения с embedded Tomcat, Jetty, Undertow
- Spring Boot предоставляет опциональные 'starter'-ы, содержащие набор совместимых зависимостей
- А так же конфигурацию кусочка контекста, который содержит в т.ч. опциональные бины
- Автоматически конфигурирует Spring где это возможно, а где не возможно, позволяет указать все настройки в одной центральной точке
- Т.е. полное нивелирование сложности конфигурирования в т.ч. своих собственных проектов
- Предоставляет production-ready features такие как метрики, самодиагностика (health check), external configuration, методы администрирования в (проект Spring Boot Actuator)

Как сделать первое приложение?

- Воспользоваться <https://start.spring.io>
- То же самое, но через IntelliJ IDEA
- Воспользоваться другим приложением как шаблоном

POM – файл, основные части.

Основные элементы

- родитель: `spring-boot-starter-parent`
- стартеры: `spring-boot-starter-*`
- плагин для сборки: `spring-boot-maven-plugin`

- общие maven-настройки
- dependencyManagement
- pluginManagement

- Наборы Maven POM-ов
- Упрощают создание Spring Boot приложений
- Содержат массу настроек по умолчанию
- Содержат все необходимые зависимости
- Spring Boot Starter-ы опциональны

Обратите внимание на зависимости, которые «приезжают» вместе со стартерами.

Стартеров много:

spring-boot-starter-*

- spring-boot-starter + spring-boot-starter-parent
- spring-boot-starter-aop
- spring-boot-starter-jdbc
- spring-boot-starter-web
- spring-boot-starter-tx
- ...

- создаёт Uber-JAR (JAR со всеми зависимостями и embedded сервлет-контейнером если подключен соответствующий стартер)
- добавляет Main-Class в манифест

- pom.xml
- spring-boot-starter-parent
- spring-boot-dependencies

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);

        // additional initialization
    }
}
```

Основные элементы:

- «запускатель» приложения *SpringApplication.run*
- аннотация *@SpringBootApplication*

«Запускатель» приложения `SpringApplication.run`

Класс для инициализации и запуска приложения Spring из `main`-метода.

Выполняет следующие действия:

- создает контекст
- обрабатывает аргументы командной строки
- обновляет контекст
- запускает `CommandLineRunner` бины

- @Configuration – это конфигурационный класс
- @EnableAutoConfiguration – сообщает Spring Boot, чтобы он добавлял бины в зависимости от того, что лежит в classpath, настроек, других бинов, и т.д.
- @ComponentScan (без параметров)
- Можно считать, что включает в себя большую часть @Enable*

@EnableWebMvc – включает Spring MVC
(Spring Boot добавляет эту аннотацию, если spring-mvc есть в classpath)

Самая интересная EnableAutoConfiguration.

Из java-doc EnableAutoConfiguration:

Enable auto-configuration of the Spring Application Context, **attempting to guess** and configure beans that you are likely to need. Auto-configuration classes are **usually applied** based on your classpath and what beans you have defined.

Аннотация запускает поиск «Auto-configuration» бинов.

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

02

Конфигурация Spring Boot приложений

@Conditional* это мощная возможность Spring Boot.

Auto-конфигурация («магия» Spring Boot) базируется на ней.

С @Conditional* Вы можете создавать бины:

- Если свойство не/задано;
- Есть класс в classpath;
- Кастомные условия.

```
@Bean
@ConditionalOnProperty(
    value = "create.socket",
    havingValue = "always"
)
@ConditionalOnClass(
    name = "ru.otus.DbDriver"
)
@Conditional(CustomCondition.class)
public MyService myService(...) {...}
```

- Любой Spring Boot Starter состоит из Auto-конфигурации и, возможно, каких-то ещё дополнительных классов (ну и зависимостей)
- Если Вы работаете в проекте с линейкой микросервисов, то самым шиком считается выпустить свою автоконфигурацию (стартер) со всеми требованиями проекта
- Ничего сложного: <https://docs.spring.io/spring-boot/docs/current/reference/html/using-boot-auto-configuration.html>
- Совсем инструкция*: <https://otus.ru/nest/post/1384/>

* до Spring Boot 2.7 включительно

```
import org.springframework.boot.autoconfigure.*;
import org.springframework.boot.autoconfigure.jdbc.*;
import org.springframework.context.annotation.*;

@Configuration
@EnableAutoConfiguration(exclude={DataSourceAutoConfiguration.class})
public class MyConfiguration {
}
```

- Может быть пустой (все нужные свойства уже заданы).
- Сюда помещаются настройки приложения
- Это фильтруемый Maven-ом ресурс.
- Новая нотация: @placeholder@ (если используете spring-boot-starter-parent).
- Да, теперь другой подход смотрим Appendix A 😊
(<https://docs.spring.io/spring-boot/appendix/application-properties/index.html>)

```
spring.datasource.url=@db.url@  
spring.datasource.username=user  
spring.datasource.password=password
```

```
#application.properties
```

```
dev.url=http://dev.bar.com  
dev.name=Developer Setup  
prod.url=http://foo.bar.com  
prod.name=My App
```

```
#application.yml
```

```
dev:  
  url: http://dev.bar.com  
  name: Developer Setup  
prod:  
  url: http://foo.bar.com  
  name: My App
```

- Yet Another Markup Language
- .yaml и .yml
- Чтобы хорошо читался машиной и человеком
- Очень удобен (особенно массивы)
- UTF-8

- Типизированные
- Иерархичные
- Массивы – одно удовольствие (особенно с YAML)
- Больше нет никаких @Value

Надо сделать следующее:

1. Подготовить сам файл (application.yml)
2. Создать класс для мапинга пропертей и отметить его аннотацией `@ConfigurationProperties(prefix = "application")`, надо указать префикс нужных пропертей в файле
3. В класс конфигурации надо добавить аннотацию `@EnableConfigurationProperties(AppProps.class)`, надо указать класс для маппинга пропертей
4. Класс с пропертями вставить как зависимость в нужный класс

<https://docs.spring.io/spring-boot/reference/features/external-config.html>

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

03

Локализация и баннер

- Находится в файле banner.txt
- Позволяет выводить некоторую информацию
- Например версию
- Может быть красивым

- Локализация/Localization (l10n) — процесс адаптации приложения, продукта или документа для удобства пользователей разных стран и культур.
- Интернационализация / Internationalization (i18n) — процесс обеспечения возможности локализации продукта.
- Имеется встроенная поддержка в Java
- Поддерживается прекрасно в Spring на уровне контекста



- Перевод сообщений осуществляется с помощью бина MessageSource и resource bundle
- В Spring его надо создать и настроить самостоятельно
- В Spring Boot только настроить
- При этом есть нюансы (по умолчанию смотрит на бандл messages + обязательно нужен дефолтный файл бандла)
- Сообщения в бандле могут быть параметризованными

- Весь пример

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

- Перенести приложение для тестирования студентов на Spring Boot и локализовать выводимые сообщения и вопросы с вариантами ответов
- Задание выполняется на базе заготовки, содержащей каркас приложения
- Необходимо дописать классы и служебные файлы заготовки, ориентируясь на оставленные в ней подсказки так, чтобы выполнялись условия задания.
- Также, решение должно удовлетворять нижеуказанным требованиям.

- То, что уже было в заготовке, должно остаться неизменным (количество, имена и текущее содержимое классов и файлов, сигнатуры методов и т.д.)
- Можно добавлять код/текст в существующие методы классов и файлы, писать свои приватные методы, а также создавать новые классы
- Вышеуказанные требования не относятся к комментариям, их после реализации нужно удалить
- Можно адаптировать/перенести актуальный код/файлы предыдущего ДЗ, но с учетом того, что некоторые классы в новой заготовке были изменены.

- Учесть актуальные требования прошлой работы
- Перенести все свойства в `application.yml`
- `MessageSource` должен быть сконфигурирован Spring Boot автоматически, на основе настроек из `application.yml`
- Локализовать выводимые сообщения и вопросы с вариантами ответов
- Сделать собственный баннер для приложения
- Перенести и адаптировать тесты
- Вместо отдельных зависимостей в файле сборки, для тестирования использовать `spring-boot-starter-test`
- В остальном, требования к тестам не меняются
- Опционально: Использовать ANSI-цвета для баннера

- Заготовка для выполнения работы:
<https://github.com/OtusTeam/Spring/tree/master/templates/hw03-spring-boot>
- Задание сдаётся только после приема предыдущей работы
- Задание сдаётся в виде ссылки на pull-request в чат с преподавателем в личном кабинете ОТУС, не в Telegram
- В pull-request должно присутствовать только то что касается текущей работы. Временные файлы и файлы IDE не должны попадать в PR.
- Инструкция по работе с pull-request-ами находится в документе "Настройка окружения.pdf" в материалах занятия.
- Вопросы можно задавать в чате ДЗ, но для оперативности рекомендуем делать это в группе Telegram.
- Код, написанный в данном ДЗ будет использоваться в дальнейших домашних заданиях первого блока курса

Вопросы?

(ставим “+” если есть вопрос и “-”
если нет)

**Пожалуйста, пройдите
опрос**

Спасибо за внимание!

Спригбута Вам!

